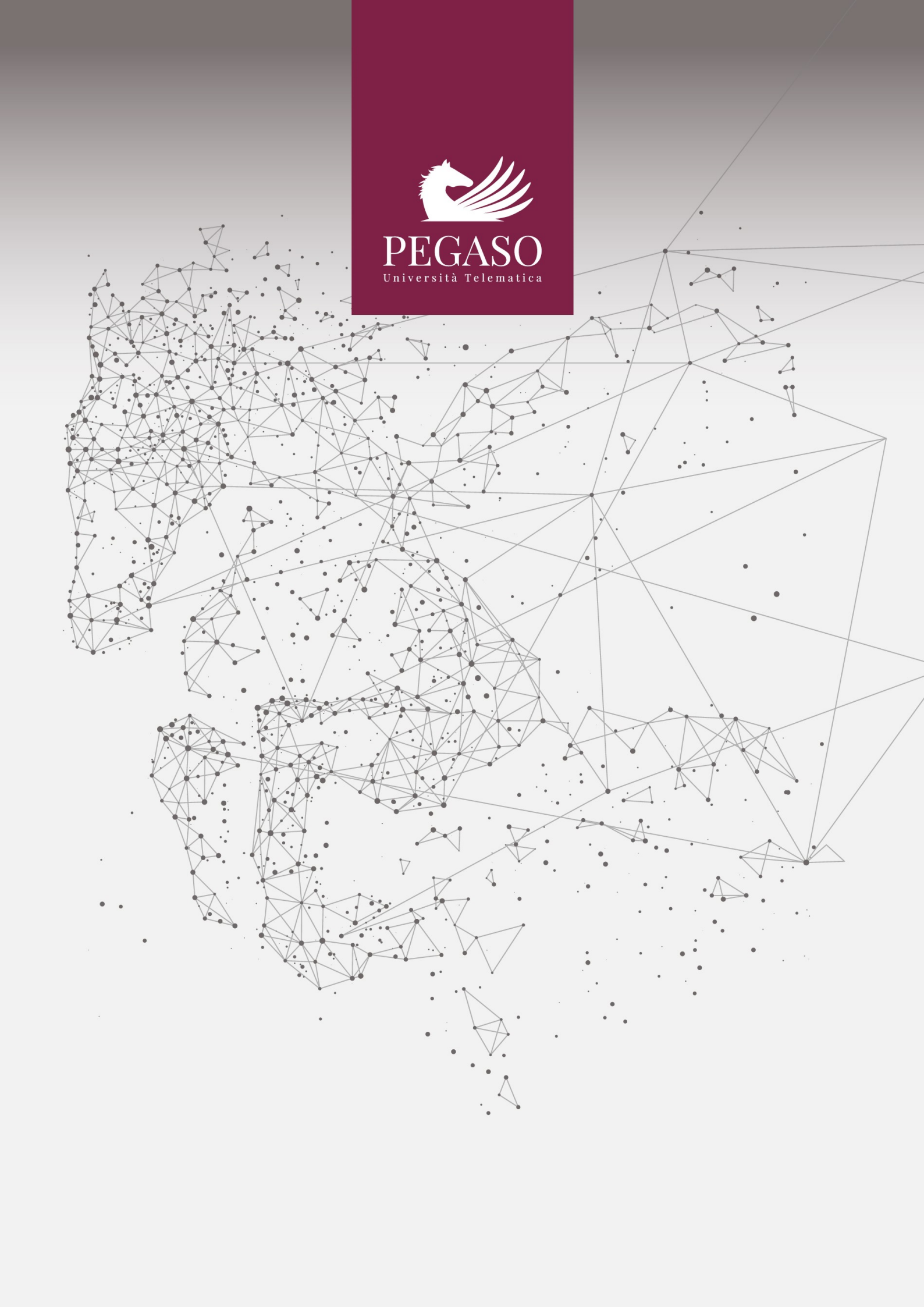

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# Indice

**1. L'EVOLUZIONE DEL WEB .....3**
**2. TECNOLOGIE NELLE VARIE FASI DEL WEB.....7**
**3. NUOVE TECNOLOGIE.....8**
**BIBLIOGRAFIA .....10**

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 1. L'evoluzione del web

Vogliamo ora analizzare l'evoluzione del web da una prospettiva più "tecnica".

Nel **1990** viene definito da Tim Berners Lee et al. l'**HTTP** (**Hyper-text transfer protocol**). Protocollo di tipo request-response pensato originariamente per l'interscambio di ipertesti. In tale contesto è fondamentale l'introduzione del concetto di risorse identificate in modo univoco sulla rete da una stringa detta **Uniform Resource Locator** (**URL**).

Successivamente viene introdotto, sempre daTim Berners Lee et al., l'**HTML** (**HyperText Markup Language**) Basato su un altro linguaggio di markup: **Standard Generalized Markup Language** (**SGML**). Questo diventa uno dei mattoni fondamentali per lo sviluppo del web come lo conosciamo oggi: è tuttora alla base della sua infrastruttura.

È un linguaggio di markup che viene interpretato da software, detti web browsers, sottoforma di documenti contenenti testo, immagini, suoni, etc. È un linguaggio di tipo dichiarativo: permette soltanto di definire la struttura dei documenti e non permette interattività. Nel contesto dei web browser, nel 1993 viene rilasciato MOSAIC, introducendo una grande innovazione: la visualizzazione di testo e immagini nella stessa pagina. Nei 3 anni successivi sono rilasciati diversi browser: Internet Explorer, Netscape Navigator, Opera.

Nel **1995** Nasce il **Javascript**, incluso per la prima volta in Netscape Navigator 2. L'HTML necessitava di un linguaggio semplice da utilizzare e che permettesse agli sviluppatori web di modificare dinamicamente il contenuto delle pagine web ed il codice Javascript veniva incluso direttamente nelle pagine tramite tag <script>. Dopo appena un anno, nasce il concetto di **CSS**, linguaggio nato per definire il modo in cui vanno visualizzati i vari elementi di un documento scritto in un linguaggio di markup: separare presentazione e contenuto. I vantaggi sono notevoli: più flessibilità, più accessibilità delle risorse, ridurre la complessità e le ripetizioni per il contenuto strutturale delle pagine web.

Nella seconda metà degli anni 90**,** le aziende iniziano a dare sempre più importanza alla presentazione dei loro contenuti: CSS e Javascript diventano tecnologie fondamentali per gli sviluppatori web per ottenere interfacce sempre più sofisticate. I contenuti non più statici ma generati dinamicamente.

È in questi anni che nasce la figura del front-end developer, ed iniziano a distinguersi gli ambiti di sviluppo sul web:

**Back-end developers**: focalizzati sulle meccaniche "dietro le quinte" di un sito web (dati, contenuti, sicurezza, struttura)

**Front-end developers**: focalizzati sulla presentazione dei contenuti e la User Experience

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

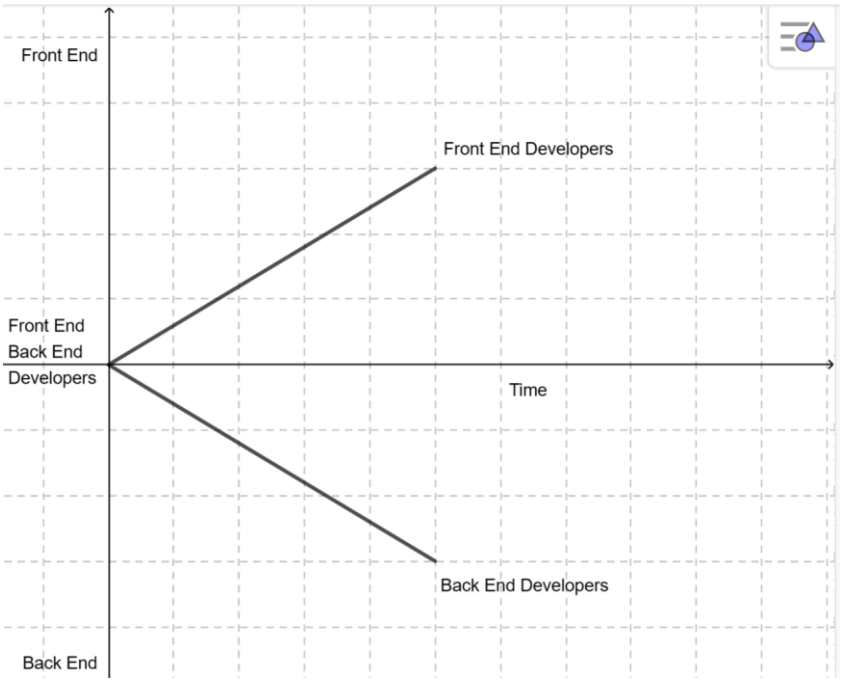



**Figura 1: Front End vs Back End**

Lo sviluppatore web pertanto doveva scegliere a-priori l'ambito in cui specializzarsi; front e backend comprendevano spesso tecnologie molto diverse.

Nel **2005**, Jesse James Garrett pubblica l'articolo "Ajax: A New Approach to Web Applications": Il termine Ajax viene coniato per descrivere una serie di tecnologie per lo sviluppo di applicazioni web che comunicano in background (in modo asincrono) con il server senza interferire con lo stato della pagina (HTML/DOM/JSON or XML/Javascript)

AJAX permette di spostare la logica dell'applicazione dal server al client (browser web) con il vantaggio di ridurre fortemente il carico di lavoro sul server, e di conseguenza i costi, pertanto:

- **Server side** → Il server gestisce la ogni aspetto dell'applicazione: Generazione contenuti, dati, validazione form utente, etc.; il server diventa molto "semplice": Pure data API o web service
- **Client side (Single-page app)** → Il client gestisce tutti gli aspetti sia di presentazione che di business logic. L'unica comunicazione con il server avviene per scambiare dati (**JSON**); il client diventa molto più "complesso": necessità di interi framework per gestire la business logic dell'applicazione e l'interattività con l'utente

L'adozione di questa tecnologia ha portato all'introduzione di molti framework JavaScript come **jQuery**, **Prototype** e **MooTools**.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

A partire dalla seconda metà degli anni **2000** e dopo l'avvento di Ajax, quindi c'è una maggiore tendenza all'astrazione, alla semplificazione ed all'introduzione di nuovi framework che ha portato via via al concetto di "**sviluppatori full-stack**": sviluppo simultaneo (spesso con stesse tecnologie) di front e backend:

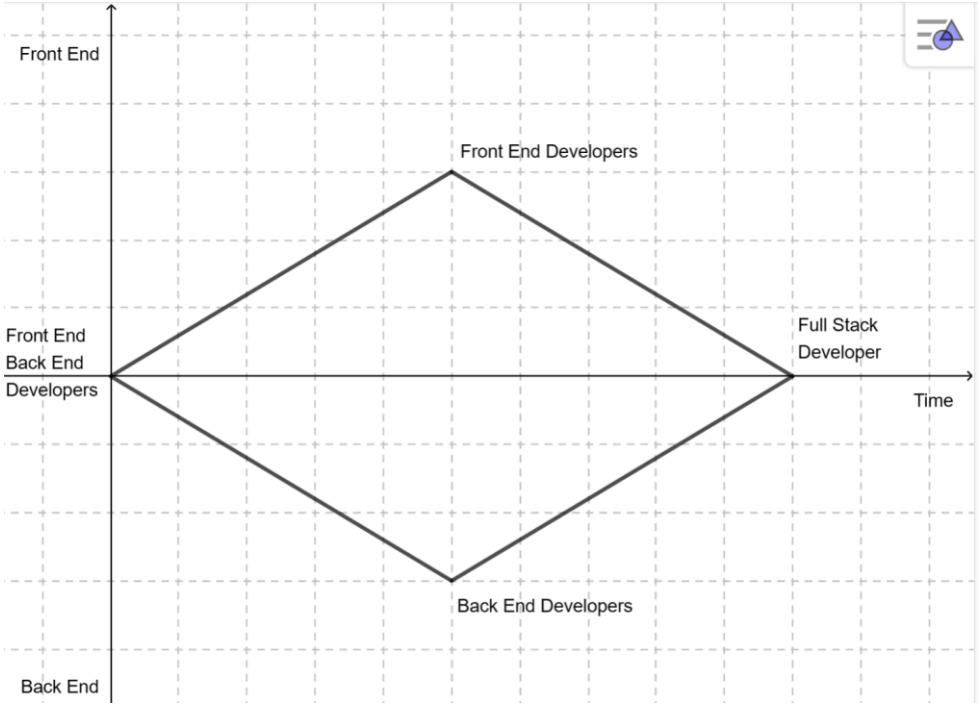



**Figura 2: Full Stack Developer**

Con l'avvento dei framework JavaScript come **AngularJS**, **React.js** e **Vue.js**, gli sviluppatori front-end hanno iniziato ad acquisire competenze back-end utilizzando JavaScript, rendendo più semplice per loro diventare full-stack developer. Allo stesso tempo, l'adozione di framework back-end come **Ruby on Rails**, **Django** e **Laravel** ha semplificato lo sviluppo lato server e ha permesso agli sviluppatori back-end di acquisire competenze front-end.

I web-frameworks moderni forniscono oggi l'interessante possibilità di non doversi necessariamente specializzare tra back-end o front-end: si può dunque seguire lo sviluppo completo di un'app web in tutte le sue parti con un chiaro aumento di produttività e di possibilità lavorative: se le stesse persone sviluppano entrambi gli aspetti (back e front end) in genere queste possono meglio interoperare.

I framework attualmente più utilizzati dai full-stack developer sono:

- **Node.js** per lo sviluppo back-end con JavaScript
- **React.js** per lo sviluppo front-end con JavaScript
- **AngularJS** per lo sviluppo front-end con JavaScript
- **Ruby on Rails** per lo sviluppo back-end con Ruby

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

- **Django** per lo sviluppo back-end con Python
- **Laravel** per lo sviluppo back-end con PHP

All'inizio degli anni **2010**, l'adozione dei dispositivi mobili ha iniziato a crescere rapidamente e questo ha portato alla creazione di siti web responsivi, in grado di adattarsi automaticamente alle dimensioni dello schermo.

In questo periodo è stato introdotto anche il concetto di Single Page Application (SPA), che consente di creare interfacce utente simili alle app native utilizzando JavaScript.

Con l'avvento delle Progressive Web App (PWA) e l'adozione di tecnologie come React Native e Ionic, gli sviluppatori hanno iniziato a creare app mobile utilizzando le stesse tecnologie utilizzate per lo sviluppo web, rendendo più facile e conveniente sviluppare per entrambe le piattaforme contemporaneamente.

Quale framework scegliere? .... Dipende, come sempre, dal contesto.

Se si volesse utilizzare uno stesso linguaggio per lo sviluppo di ogni aspetto di una web application, dal **2013** ci sarebbe l'opzione **MEAN**: uno stack di tecnologie JavaScript usate per programmare applicazioni web dinamiche.

Le tecnologie che compongono MEAN sono **MongoDB**, **Express**, **Angular** e **Node**.

- Angular è il framework di scripting client-side su JavaScript
- Node è l'ambiente di runtime server-side per eseguire JavaScript
- Express il framework di scripting server-side
- MongoDB il database NoSQL

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 2. TECNOLOGIE NELLE VARIE FASI DEL WEB

Il web è stato attraverso diverse fasi di evoluzione, ciascuna delle quali è stata caratterizzata da tecnologie di sviluppo specifiche.

Il web 1.0, intorno agli anni '90, era composto principalmente da siti web statici scritti in HTML e utilizzando solo pochi script lato client scritti in JavaScript. In questa fase, le tecnologie di sviluppo utilizzate erano principalmente HTML, CSS e JavaScript.

Il web 1.0 è il web dell'***Only Read Web***

Il web 2.0, intorno agli anni 2000, ha visto l'adozione di nuove tecnologie come l'Ajax e i framework JavaScript come jQuery, Prototype e MooTools. Queste tecnologie hanno permesso di creare interfacce utente più interattive senza dover ricaricare la pagina intera. In questa fase sono state introdotte anche tecnologie come PHP e Ruby on Rails per lo sviluppo back-end, e database come MySQL e PostgreSQL.

Il web 2.0 è il web del ***Read-Write Web***

Il web 3.0, intorno al 2010, è stato caratterizzato dall'adozione di dispositivi mobili e dalla crescente importanza dei dati e delle connessioni. In questa fase sono state introdotte le Progressive Web App (PWA) e sono stati utilizzati framework come AngularJS, React.js e Vue.js per lo sviluppo front-end, mentre sono stati utilizzati database NoSQL come MongoDB e Cassandra per lo sviluppo back-end.

Il web 3.0 è il web del ***Read-Write-Execute Web***

Il web 4.0, attualmente in fase di sviluppo, è caratterizzato dall'intelligenza artificiale, dalla realtà virtuale e aumentata, dalla blockchain e dalla Internet of Things (IoT). In questa fase si stanno utilizzando tecnologie come GraphQL per gestire le richieste dei dati, si sta utilizzando la programmazione funzionale per sviluppare applicazioni, si stanno utilizzando tecnologie come gRPC e WebAssembly per migliorare le prestazioni, e si stanno utilizzando tecnologie blockchain per la sicurezza e la trasparenza dei dati.

Da precisare che queste fasi sono indicative e non sono state formalmente stabilite, ogni azienda o sviluppatore potrebbe avere una propria visione della suddivisione e delle tecnologie utilizzate.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 3. NUOVE TECNOLOGIE

È necessario parlare di cosa è emerso a partire dalla seconda metà degli anni 2000: l'avvento del **cloud**. Nel **2006** Amazon ha lanciato Amazon Web Service (AWS), e rapidamente altri player si sono immessi nel mercato: Google ed IBM in primis.

Il cloud computing si basa principalmente sulla nozione di servizio. I servizi sono componenti computazionali autonomi, indipendenti dalla piattaforma, che possono essere pubblicati e composti ottenendo reti di applicazioni distribuite. L'applicazione così ottenuta diventa a sua volta un servizio disponibile sulla rete.

L'essenza del cloud computing è **X as a Service**, che può essere tradotto con "**X come Servizio**".

I servizi cloud forniscono agli sviluppatori una serie di strumenti e servizi, come server virtuali, archiviazione, banche dati e sistemi di gestione delle applicazioni, che possono essere utilizzati per creare e distribuire applicazioni web.

Ci sono diversi vantaggi nello sviluppo web sul cloud.

In primo luogo, consente agli sviluppatori di concentrarsi sullo sviluppo delle loro applicazioni, invece di preoccuparsi della gestione dell'infrastruttura sottostante.

In secondo luogo, i servizi cloud sono scalabili e possono essere adattati alle esigenze cambianti delle applicazioni, eliminando la necessità di investire in hardware costoso.

Inoltre, i servizi cloud sono generalmente accessibili ovunque ci sia una connessione internet, il che consente agli sviluppatori di lavorare da qualsiasi luogo.

Altra innovazione importante apparsa intorno alla prima metà degli anni 2010 è quella dei **container**.

I **container** sono simili alle macchine virtuali, ma invece di creare un intero sistema operativo virtuale, i container condividono il kernel del sistema operativo host e utilizzano solo le risorse di cui hanno bisogno. Ciò li rende più leggeri e più efficienti rispetto alle macchine virtuali.

I container vengono utilizzati per creare ambienti di sviluppo e test coerenti, per la distribuzione delle applicazioni e per la gestione del ciclo di vita delle applicazioni. Inoltre, i container possono essere utilizzati per creare cluster di più container che lavorano insieme per creare un'applicazione distribuita.

Docker è uno dei principali fornitori di container e fornisce un sistema per creare, distribuire e gestire container. Altri fornitori di container includono Kubernetes, OpenShift e Rancher.

Docker è stato creato come un modo per semplificare la creazione e la distribuzione di applicazioni, utilizzando container per isolare le risorse del sistema operativo e le dipendenze dell'applicazione. Docker

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

ha introdotto un formato di immagine container standardizzato e un sistema di gestione dei container che ha reso facile per gli sviluppatori creare e distribuire le loro applicazioni in modo che possano essere eseguite su qualsiasi sistema.

In generale, l'utilizzo dei container consente agli sviluppatori di creare e distribuire facilmente le loro applicazioni in modo che possano essere eseguite su qualsiasi sistema, riducendo i problemi di compatibilità e facilitando la distribuzione e la gestione delle applicazioni.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# Bibliografia


- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010;
- S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.


PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*